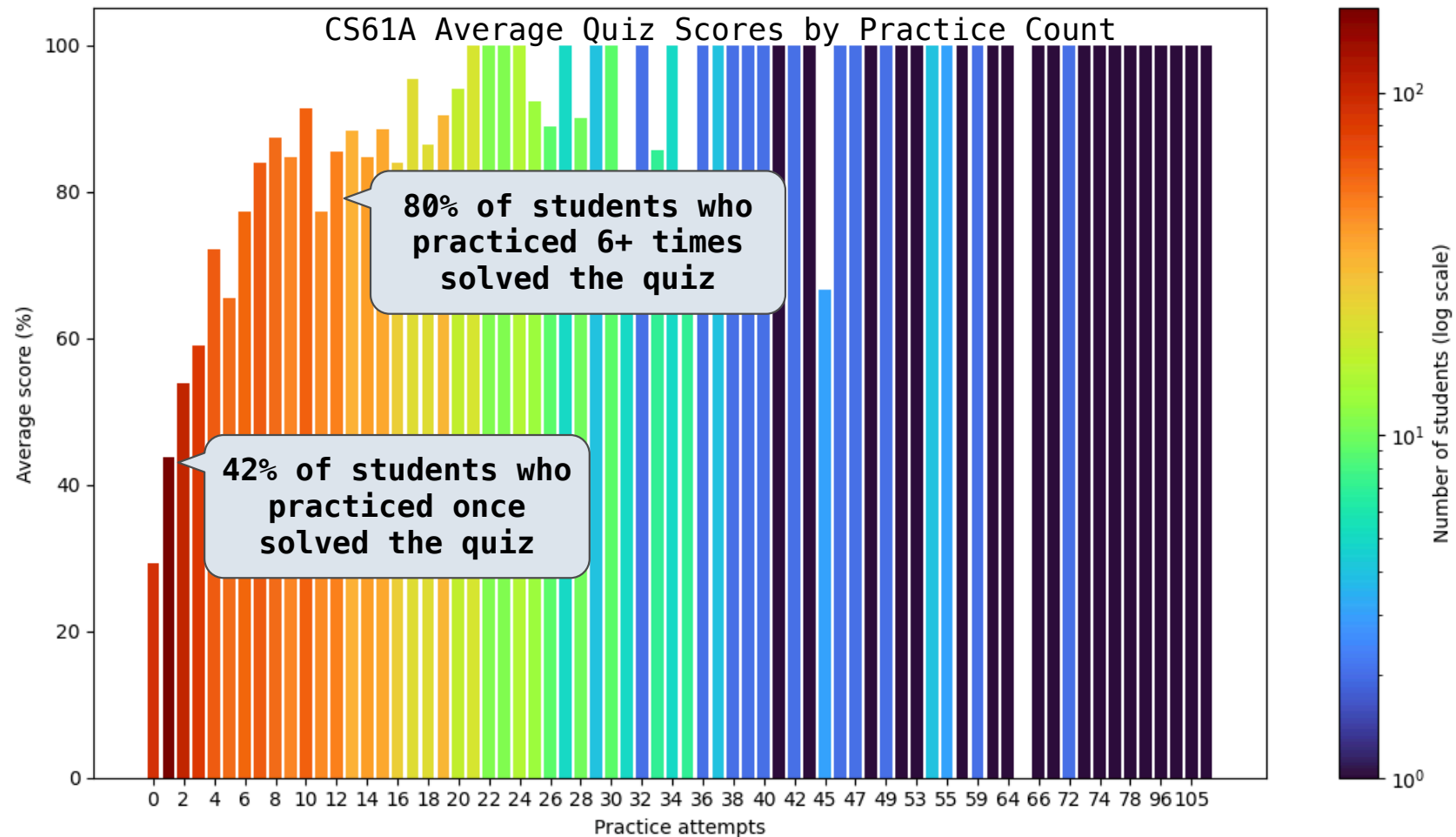


Objects

Announcements

Practice Repeatedly for the Recursion Quiz!



Dictionaries

```
{'Dem': 0}
```

Dictionary Comprehensions

`{<key exp>: <value exp> for <name> in <iter exp> if <filter exp>}`

Short version: `{<key exp>: <value exp> for <name> in <iter exp>}`

Example: Multiples

Implement **multiples**, which takes two lists of positive numbers **s** and **factors**. It returns a dictionary in which each element of **factors** is a key, and the value for each key is a list of the elements of **s** that are multiples of the key.

```
def multiples(s, factors):  
    """Create a dictionary where each factor is a key and each value  
    is the elements of s that are multiples of the key.  
  
    >>> multiples([3, 4, 5, 6, 7, 8], [2, 3])  
    {2: [4, 6, 8], 3: [3, 6]}  
    >>> multiples([1, 2, 3, 4, 5], [2, 5, 8])  
    {2: [2, 4], 5: [5], 8: []}  
    """"  
  
    return {d: [x for x in _____ if _____] for d in _____}
```

Example: Multiples

Implement **multiples**, which takes two lists of positive numbers **s** and **factors**. It returns a dictionary in which each element of **factors** is a key, and the value for each key is a list of the elements of **s** that are multiples of the key.

```
def multiples(s, factors):  
    """Create a dictionary where each factor is a key and each value  
    is the elements of s that are multiples of the key.  
  
    >>> multiples([3, 4, 5, 6, 7, 8], [2, 3])  
    {2: [4, 6, 8], 3: [3, 6]}  
    >>> multiples([1, 2, 3, 4, 5], [2, 5, 8])  
    {2: [2, 4], 5: [5], 8: []}  
    """"  
  
    return {d: [x for x in s if x % d == 0] for d in factors}
```

Type Hints

Type Hints Annotate Functions

Formal parameters and return values can be annotated with type hints

Type hints **do not** restrict what values can be arguments or return values for the function

Union types are created with `|` and indicate that a value should be either of two types

Type aliases give names to types, typically to union types

```
def square(x: int) -> int:  
    return x * x
```

```
>>> square(5)  
25  
>>> square(1.4142)  
1.9999616399999998
```

Type checker says it's bad,
but it's actually fine

```
def double(x: str | float) -> str | float:  
    if isinstance(x, str):  
        return str(2 * float(x))  
    else:  
        return 2 * x
```

```
type Quantity = str | float  
def double(x: Quantity) -> Quantity:  
    if isinstance(x, str):  
        return str(2 * float(x))  
    else:  
        return 2 * x
```

You won't be tested
on how to write
type hints.

They are meant to
help you.

Code that works but
has type checking
errors is fine.

Data Classes

Objects

Python is an object-oriented language

- Objects represent information
- They consist of data and behavior, bundled together to create abstractions

Dictionaries are objects:

- A correspondence between keys and values is what a dict represents: `d = {3: 9, 4: 16}`
- `d.get(4)` (which evaluates to 16) is part of how a dict behaves
- `get` is called a method: it's a function invoked on the dictionary
- Its type is `dict`
- When you print a dict, the format looks like a dict

All values in Python are objects: each has data as well as behavior determined by its type

Class Statements Create User-Defined Types

A class statement creates a new type; the **@dataclass** decorator simplifies class statements

In a data class, each name with a type hint creates an attribute for instances of the type

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Line:
```

```
    slope: float
```

```
    intercept: float
```

```
    def __str__(self):
```

```
        return format_line(self)
```

`__str__` determines what happens when you call **print** or **str** on a **Line**

```
def parallel(c: Line, d: Line) -> bool:
```

```
    return c.slope == d.slope
```

```
def format_line(c: Line) -> str:
```

```
    return 'y = ' + str(c.slope) + 'x + ' + str(c.intercept)
```

```
    OR return f'y = {c.slope}x + {c.intercept}'
```

```
>>> c = Line(3, 4)
```

```
>>> c.slope
```

```
3
```

```
>>> c.intercept
```

```
4
```

```
>>> c
```

```
Line(slope=3, intercept=4)
```

```
>>> type(c) == Line
```

```
True
```

```
>>> isinstance(c, Line)
```

```
True
```

```
>>> print(c)
```

```
y = 3x + 4
```

Partitions with Restrictions

Count Partitions Review

count_partitions(n: int, m: int) -> int:

Count the ways to make n by summing positive parts up to m in increasing order

count_partitions(12, 4) *15 + 19 = 34 ways to make 12 using parts up to 4*

$$4 + 4 = 12$$

$$\dots = 12$$

$$2+3+3+4 = 12$$

$$\dots = 12$$

$$1+1+1+1+1+1+1+4 = 12$$

$$3+3+3+3 = 12$$

$$1+2+3+3+3 = 12$$

$$\dots = 12$$

$$1+1+1+1+1+1+1+1+1+1+1+1 = 12$$

*15 ways to make 12 using at least one 4: **count_partitions(12-4, 4)***

*19 ways to make 12 using parts up to 3: **count_partitions(12, 4-1)***

Strategy: Use a part of size m; recurse for the rest

AND

Don't use any parts of size m; recurse for the rest

Count Partitions Review

`count_partitions(n: int, m: int) -> int` abbreviated to **cp**

cp(n, m): count the ways to make n by summing positive parts up to m in increasing order

```
def cp(n, m):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
    return cp(n-m, m) + cp(n, m-1)
```

cp(4, 2) ➡ 3 ways

choose at least 1 m=2

cp(4-2, 2) ➡ 2 ways

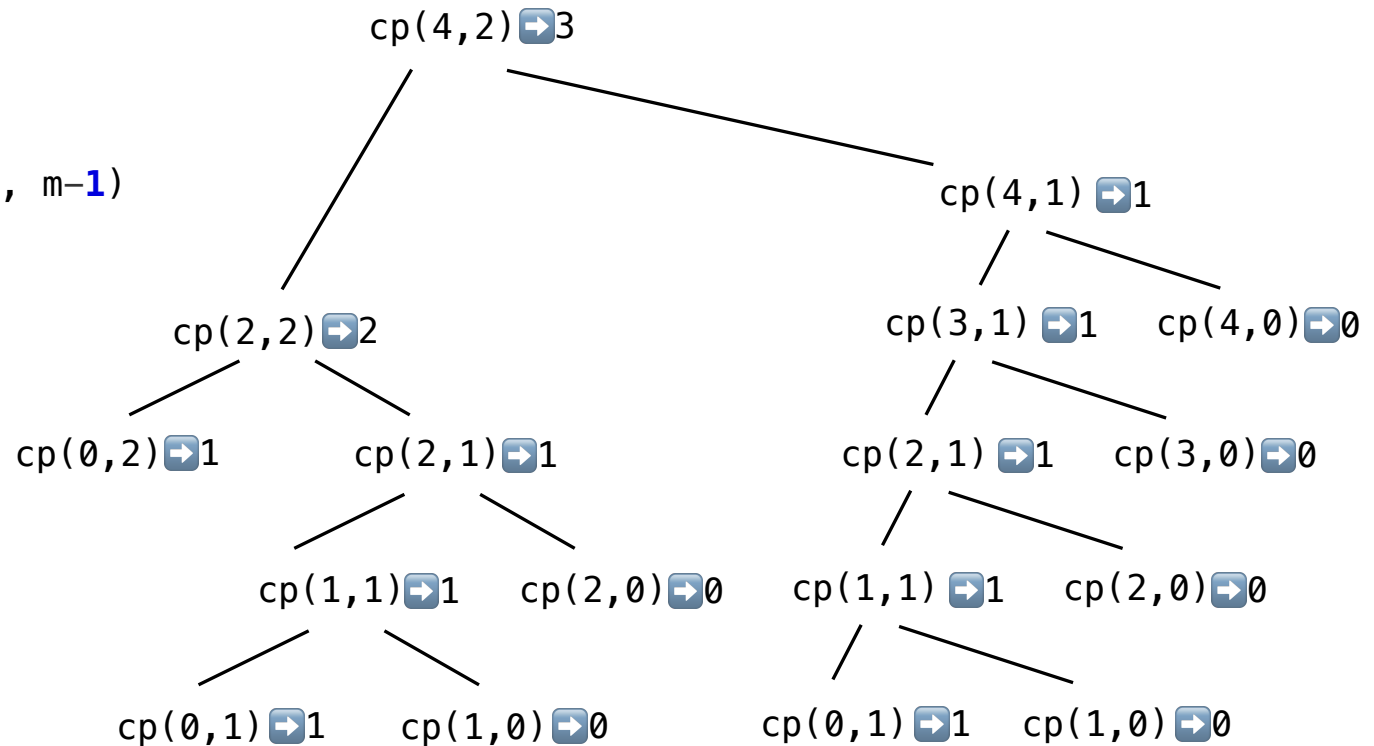
2+2 = 4

1+1+2 = 4

choose no more m=2:

cp(4, 1) ➡ 1 way

1+1+1+1 = 4



Count Partitions Alternative Implementations

`count_partitions`

`cp(n, m)`: count the ways to make n by summing positive parts up to m in increasing order

```
def cp(n, m):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
    return cp(n-m, m) + cp(n, m-1)
```

You can add base cases

```
elif m == 1:  
    return 1
```

You can sometimes make base cases unnecessary by skipping recursive calls

`cp(4, 2)` ➡ 3 ways

choose at least 1 $m=2$

`cp(4-2, 2)` ➡ 2 ways

$2+2 = 4$

$1+1+2 = 4$

choose no more $m=2$:

`cp(4, 1)` ➡ 1 way

$1+1+1+1 = 4$

```
def cp(n, m):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
    if m > n:  
        return cp(n, n)  
    return cp(n-m, m) + cp(n, m-1)
```

Tracking Constraints

count_partitions

`cp(n, m)`: count the ways to make n by summing positive parts up to m in increasing order using **at most 3 total parts** in the sum: $2 + 2 + 2$ is fine, but $1 + 1 + 2 + 2$ is not

Original implementation:

```
def cp(n, m):
    if n == 0:
        return 1
    elif n < 0 or m == 0:
        return 0
    return cp(n-m, m) + cp(n, m-1)
```

There is no way to add a base case for using too many parts, because we don't know how many parts we've used.

Revised implementation:

```
def cp_3total(n, m):
    return cpk(n, m, 3)

def cpk(n, m, k):
    if n == 0:
        return 1
    elif n < 0 or m == 0:
        return 0
    elif k == 0:
        return 0
    return cpk(n-m, m, k-1) + cpk(n, m-1, k)
```

At most k parts total

We used a part, so $k-1$ remain

Lesson: To add a constraint, add arguments that track whether the constraint was violated

Discussion Question

count_partitions

cp(n, m): count the ways to make n by summing positive parts up to m in increasing order using **each part at most 3 times**: 1 + 1 + 2 + 2 is fine, but 1 + 1 + 1 + 1 + 2 is not

Original implementation:

```
def cp(n, m):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
    return cp(n-m, m) + cp(n, m-1)
```

There is no way to add a base case for using too many parts, because we don't know how many parts we've used.

Revised implementation:

```
def cp_3_of_each_part(n, m):  
    return cpk(n, m, 3)
```

```
def cpk(n, m, k):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
  
    no_m = cpk(____, _____, _____)  
    if k == 0:  
        return no_m  
    return cpk(n-m, m, _____) + no_m
```

At most k of m & at most 3 of each smaller parts

Lesson: To add a constraint, add arguments that track whether the constraint was violated

Discussion Question

count_partitions

cp(n, m): count the ways to make n by summing positive parts up to m in increasing order using **each part at most 3 times**: 1 + 1 + 2 + 2 is fine, but 1 + 1 + 1 + 1 + 2 is not

Original implementation:

```
def cp(n, m):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
    return cp(n-m, m) + cp(n, m-1)
```

There is no way to add a base case for using too many parts, because we don't know how many parts we've used.

Revised implementation:

```
def cp_3_of_each_part(n, m):  
    return cpk(n, m, 3)
```

```
def cpk(n, m, k):  
    if n == 0:  
        return 1  
    elif n < 0 or m == 0:  
        return 0  
  
    no_m = cpk(n, m-1, k)  
    if k == 0:  
        return no_m  
    return cpk(n-m, m, k-1) + no_m
```

At most k of m & at most 3 of each smaller parts

Lesson: To add a constraint, add arguments that track whether the constraint was violated